Project Executable Files

Date	14 July 2026
Team ID	RW-2026-01
Project Name	Rising Waters – AI-Based Flood Prediction System
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

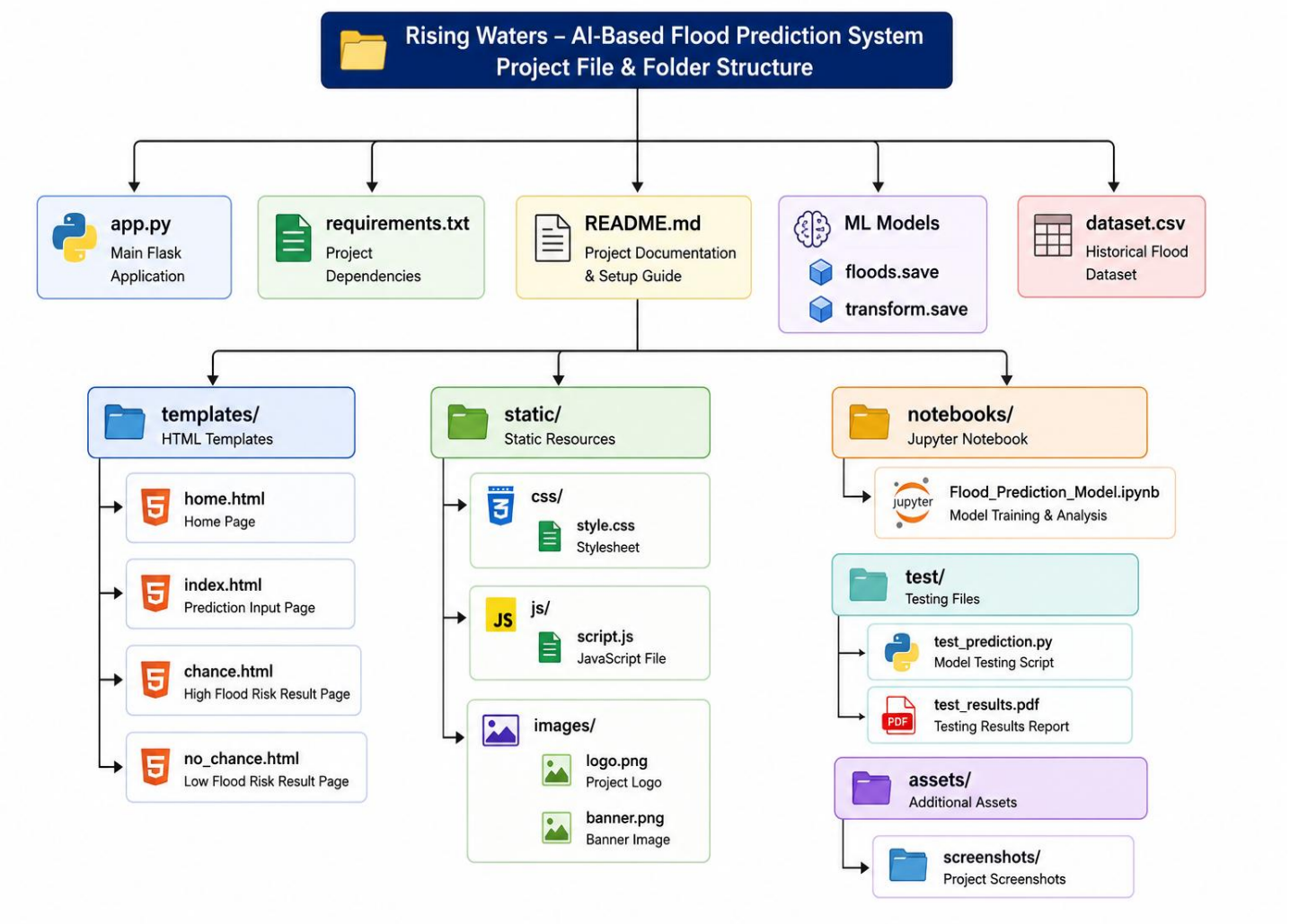
S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide	Yes
3	requirements.txt	Yes
4	Database schema / Seed files	NA
5	Environment configuration file (.env.example)	**No** *(Not required for local Flask application)*
6	Deployed application URL	**No** *(Local deployment)*
7	Executable Binary / APK	NA
8	Docker file / Containerization	No
9	Test files and test results	Yes
10	Demo video / Walkthrough	Yes

Step 2: File / Folder Structure

Overview:

The **Rising Waters** project follows a modular file and folder structure to improve readability, maintainability, and scalability. The project is organized into separate directories for application code, machine learning models, datasets, HTML templates, static resources, testing files, and documentation. This organization enables efficient development, testing, deployment, and future enhancements while ensuring that all project components are easy to locate and manage.

Project File & Folder Structure:



Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Local Deployment (`http://127.0.0.1:5000`)
Login Credentials (Demo)	Not Applicable
Platform / Hosting Provider	Flask Local Development Server
Repository Link	https://github.com/238x1a0414-eng/Rising-Waters
Demo Video Link	

Step 4: Run Instructions

Instructions to Run the Project Locally

Prerequisites

Before running the project, ensure the following software is installed on your system:

- Python 3.10 or later
- Visual Studio Code (or any Python IDE)
- Git (optional, for cloning the repository)
- A modern web browser (Google Chrome, Microsoft Edge, or Mozilla Firefox)

Step 1: Download the Project

- Download the project folder from the GitHub repository or obtain the project ZIP file.
- Extract the ZIP file (if applicable) and open the **Rising-Waters** project folder.

Step 2: Open the Project

- Open **Visual Studio Code**.
- Select **File → Open Folder**.
- Choose the **Rising-Waters** project directory.

Step 3: Install Required Libraries

Open the integrated terminal and execute the following command:

pip install -r requirements.txt

This installs all required Python packages, including Flask, Pandas, NumPy, Scikit-learn, and XGBoost.

Step 4: Verify Project Files

Ensure the following files are available in the project directory:

- app.py
- requirements.txt
- dataset.csv
- floods.save
- transform.save
- templates/
- static/

Step 5: Run the Application

Execute the following command in the terminal:

python app.py

The Flask server will start successfully.

Example output:

* Running on http://127.0.0.1:5000/

* Debug mode: On

Step 6: Open the Web Application

Open any web browser and navigate to:

http://127.0.0.1:5000

The **Rising Waters – AI-Based Flood Prediction System** home page will be displayed.

Step 7: Test the Flood Prediction

1. Enter the required weather parameters.
2. Click the **Predict** button.
3. The application processes the input using the trained XGBoost model.
4. The system displays the flood prediction result (High Flood Risk or Low Flood Risk) along with the corresponding alert.

Step 8: Stop the Application

After testing, return to the terminal and press:

Ctrl + C

to stop the Flask server.

Expected Output

- The web application launches successfully.
- User inputs are accepted without errors.
- The machine learning model generates accurate flood risk predictions.

- Results are displayed on the prediction page.
- The application runs smoothly without runtime errors.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Real-time weather API integration is not implemented.	Uses historical dataset for prediction; live API can be added in future versions.
2	Local deployment only.	Can be deployed to Render, AWS, Azure, or Heroku for public access.
3	Alert notifications are displayed only in the web application.	SMS and Email notifications can be integrated as future enhancements.

Outcome:

The Rising Waters project includes all essential executable files, source code, machine learning models, documentation, and setup instructions required to run and evaluate the application successfully. The organized project structure enables straightforward deployment, testing, and future enhancements.